ExplainBack — GitHub Workflow & Team Rules

UI-only V0 • Three-person team • Working agreement

Keep main stable and develop each task on its own branch. Team members merge through pull requests with one teammate's approval. The owner, DuyHuynh3010, may bypass this ruleset's review and check requirements when merging a pull request.
Scope: The current version covers the user interface and mock data. Coordinate with the team before adding backend services, AI APIs, authentication, or other new scope.

Ruleset setup check

The initial JSON establishes standard protection. An owner-only pull-request bypass has now been added on GitHub for DuyHuynh3010. The original JSON file does not include this exception.

1. **Save and activate the ruleset.** Confirm that it is saved with Enforcement status set to Active.
2. **Check the target and access.** Target main and add only the user DuyHuynh3010 to the bypass list with "For pull requests only." Other contributors keep the standard review requirements. GitHub Free supports rulesets on public repositories; private repositories require an eligible plan.
3. **Allow squash merging.** Open Settings → General → Pull Requests and enable Allow squash merging. The supplied JSON allows only squash merges.
4. **Test the review requirement.** For a non-owner contributor, merging should be blocked before the required approval. DuyHuynh3010 may use the bypass option to merge a pull request without that approval or checks required by this ruleset. Merge conflicts must still be resolved.

The supplied ruleset does not require CI checks yet. The team is responsible for checking the build and testing the relevant behavior before merging.

Daily development workflow

1. **Choose one clear task before coding.** For example: "Build the learning source input UI." Tell the team which part you are implementing and which files you expect to change so that two people do not duplicate the same work.
2. **Check your working tree.** Run git status. If you have uncommitted changes, save them in commits on the branch where you are working before switching branches. Do not discard changes just to make switching easier.
3. **Update main and create a task branch.** Switch to main, pull the latest version with --ff-only, and create a branch such as feat/source-input, feat/recorder-preview, or fix/mobile-layout. See the command reference below.

4. **Implement only the agreed scope.** When using AI coding tools, specify the task, the files they may change, and the behavior they must preserve. Discuss changes to shared types, page.tsx, global CSS, or dependencies with the team first.
5. **Inspect and stage the intended files.** Run git status and git diff. In VS Code Source Control, click + beside each file you want to include. Then run git diff --cached to inspect the staged changes. Commit only changes you understand and intend to include.
6. **Write a specific commit message.** Use a clear prefix and description, such as feat: add source input UI. You may commit multiple times on your feature branch to save progress. If the feature is unfinished, describe its status and keep its pull request in Draft.
7. **Sync with main before finalizing the pull request.** After committing your current work, run git fetch origin and git merge origin/main on your feature branch. If conflicts occur, resolve them with the teammate responsible for the affected code, then stage and commit the resolution. Test the combined changes again.
8. **Test the feature and push your branch.** Run the app, check the affected screens, and test the demo flow. For a project using npm with a build script, run npm run build and resolve build failures before marking the PR ready. Use git push -u origin feat/source-input for the first push; later pushes on that branch can use git push.
9. **Open a pull request into main and request a review.** Describe what changed, how to test it, any unfinished work, and include screenshots for UI changes. Other contributors need one approval and a new approval after reviewable changes. The owner, DuyHuynh3010, may instead use the bypass option and Squash and merge without waiting for this ruleset's approvals or checks. A pull request is still required; this exception does not permit direct pushes to main.
10. **Start the next task from the updated main branch.** After merging, switch to main and pull the latest changes. Create a new branch for the next task instead of continuing on the branch that was just squash-merged.

Commit message prefixes

feat: add a feature; fix: correct a bug; style: change the UI; refactor: reorganize code without changing behavior; docs: update documentation; chore: maintain configuration or tooling. These prefixes are team conventions. The supplied JSON does not automatically enforce commit message wording.

What not to do when committing, pushing, or merging

1. **Do not commit project work on main or push directly to main.** GitHub rules do not prevent local commits on that branch, so each teammate must check the active branch before working.
2. **Do not commit secrets.** This includes API keys, access tokens, passwords, .env files, and account credentials. Use placeholder values only in .env.example.

3. **Do not commit generated or unnecessary files.** Exclude node_modules/, .next/, dist/, caches, and unnecessary logs. Commit the project's package-manager lockfile, such as package-lock.json.
4. **Do not run git add . and commit without reviewing the result.** Prefer staging individual files and reading git diff --cached before committing.
5. **Do not combine unrelated tasks in one commit or pull request.** A PR that fixes the Record button should not also restructure the entire application.
6. **Do not use vague commit messages.** Avoid messages such as update, fix stuff, done, or abc. Describe the actual change.
7. **Do not submit AI-generated code without reading and testing it.** The PR author is responsible for every change included in the pull request.
8. **Do not overwrite a teammate's work or blindly accept one side of a merge conflict.** Understand which changes must be preserved and coordinate with the relevant teammate.
9. **Do not force-push to a shared branch or run git reset --hard without understanding what will be lost.** This workflow normally does not require either operation.
10. **Do not treat review or testing as a formality.** Other contributors must resolve build failures before merging. DuyHuynh3010 may bypass the ruleset's requirements; basic testing remains recommended for everyone. CI is not currently required by this ruleset.
11. **Do not change the package manager, delete the lockfile, or add major dependencies without discussion.** Everyone should use the same setup and run commands.
12. **Do not disable the ruleset or extend bypass access without the owner's approval.** DuyHuynh3010 is the designated exception and may bypass this ruleset when merging a pull request. All other contributors follow the standard requirements.

Command reference

Run these commands from the project directory, one at a time. Stop and resolve errors before continuing. Replace feat/source-input with the branch name for your own task.

Start a task — workflow steps 2–3

Run git status first. Continue only after saving any existing work on the appropriate branch.

```
git status
git switch main
git pull --ff-only origin main
git switch -c feat/source-input
```

Review and commit — workflow steps 5–6

Inspect your changes, stage the intended files in VS Code, and review the staged diff before committing.

```
git status
git diff
```

```
git diff --cached
git commit -m "feat: add source input UI"
```

Sync, test, and push — workflow steps 7–8

Run these on your feature branch after committing current work. Resolve and commit any merge conflicts before proceeding. Use the build command only if the project uses npm and defines a build script.

```
git fetch origin
git merge origin/main
npm run build
git push -u origin feat/source-input
```

After merging — workflow step 10

```
git switch main
git pull --ff-only origin main
```

References

[GitHub Docs — Creating rulesets for a repository]
[GitHub Docs — Available rules for rulesets]
[GitHub Docs — Configuring commit squashing for pull requests]
[GitHub Docs — GitHub flow]